**LOVELY PROFESSIONAL UNIVERSITY**

**B. TECH**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

# MINI PROJECT REPORT

**ON**

# SMARTINVENTORY: A SECURE, HIGH-PERFORMANCE INTERNAL WAREHOUSE & INVENTORY MANAGEMENT SYSTEM

Submitted in partial fulfilment of the requirements for

**BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING**

**Submitted by**

Shubham Kumar — Registration No.: 12315449

Tapan Kumar Mahato — Registration No.: 12315468

**Under the guidance of**

Ankit Galhotra

**7th Semester / Academic Year 2026–2027**

## Certificate (if required by the department)

This is to certify that the mini project titled "SmartInventory: A Secure, High-Performance Internal Warehouse & Inventory Management System" has been developed and submitted by Shubham Kumar (Registration No.: 12315449), Tapan Kumar Mahato (Registration No.: 12315468) of Bachelor of Technology in Computer Science & Engineering during 7th Semester / Academic Year 2025–2026, under the guidance of Ankit Galhotra. The work is submitted for academic evaluation in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering.

The project represents bonafide practical engineering work addressing real-world warehouse challenges—specifically atomic multi-document transaction isolation, negative stock prevention, and server-side MongoDB aggregation pipelines. To the best of our knowledge, the results embodied in this report have not been submitted for any other degree or qualification.

**Guide signature: _____**

**Project coordinator / HoD signature: _____**

**Date and place: _____**

## Declaration by Students

We declare that the project titled "SmartInventory: A Secure, High-Performance Internal Warehouse & Inventory Management System" is our work carried out for academic purposes. All external sources, libraries, datasets, code examples and documentation used have been acknowledged. Any reused or adapted material is identified in the report.

We affirm that the database schema designs, Express API route handlers, Mongoose transactional boundaries, and React front-end components described in this report were developed and verified by us for this project. We have complied with all academic integrity and professional ethics standards throughout this endeavor.

**Student name(s) and signature(s):**

1. Shubham Kumar (Roll No. 12315449) : Shubham Kumar
2. Tapan Kumar Mahato (Roll No. 12315468) : Tapan Kumar Mahato

**Date: 25 October 2026**

## Acknowledgement

I would like to express my sincere gratitude to my project guide and professors in the Department of Computer Science & Engineering. Their practical feedback on database design, session locking models, and server-side data aggregation was instrumental in helping me build this system cleanly and securely.

I am also grateful to the department and university for providing the computer laboratory facilities, fast internet access, and testing environment that supported the steady development of this project.

Special thanks go to my friends and peers who took the time to test the application, enter mock stock items, and give honest feedback on UI usability. That input directly helped fix several tricky interface issues, such as multi-tab state syncing and clear validation alerts. Finally, I thank my family for their constant encouragement throughout my studies.

## Abstract

SmartInventory is an internal warehouse operations tool built using the MERN stack (MongoDB, Express, React 19, and Node.js). It was developed to solve common and frustrating problems in internal stock control—such as spreadsheet race conditions, phantom inventory counts, and the complete lack of an audit trail. Rather than building a generic retail e-commerce site with shopping carts and payment checkouts, this project focuses strictly on warehouse floor efficiency and rock-solid data integrity.

The core system provides two-layer negative stock prevention (both in the React UI and inside atomic backend controllers), an immutable movement journal that records every incoming and outgoing box, and multi-document ACID transactions that roll back cleanly if anything fails. To keep the interface snappy on lower-spec warehouse devices, dashboard totals and low-stock alerts are calculated directly on MongoDB Atlas using native C++ aggregation pipelines ($facet, $match, $group), cutting payload sizes by 99.6%. Security is implemented through salted BCrypt password hashing, Google OAuth 2.0 Single Sign-On, Helmet headers, and NoSQL injection sanitizers. All 8 core test cases passed with zero data discrepancies.

**Keywords :** *MongoDB Atlas, Inventory Management, MERN Stack, Aggregation Pipeline, ACID Transactions, Role-Based Access Control.*

## Table of Contents


| Chapter / Section Description | Page |
|---|---|
| Certificate & Student Declaration | Page 2–3 |
| Acknowledgement & Abstract | Page 4–5 |
| Chapter 1: Introduction (Overview, Background, Scope, Objectives) | Page 7 |
| Chapter 2: Problem Statement & Proposed Solution (Benchmark & Roles) | Page 7 |
| Chapter 3: Requirement Analysis (Functional, NFR, Hardware, Tech Stack) | Page 8 |
| Chapter 4: System Design and Architecture (Architecture & Workflow) | Page 9 |
| Chapter 5: Database Design Using MongoDB (Schema, Sample JSON, Indexes) | Page 10 |
| Chapter 6: Implementation & Application Screenshots | Page 11 |
| Chapter 7: Testing, Results and Discussions | Page 12 |
| Chapter 8: Conclusion and Future Scope | Page 13 |
| References | Page 13 |
| Appendices A, B, C (Code, Extra Views, Setup) | Page 14–15 |


## List of Figures and Tables


- Figure 4.1: Three-Tier Decoupled System Architecture and Security Middleware Pipeline
- Figure 4.2: Atomic Stock Transaction Lifecycle and Negative Stock Prevention Flowchart
- Figure 5.1: MongoDB Relational Entity-Relationship Diagram and Foreign-Key Linkages
- Figure 6.1–6.6: Core Application Views (Login, Dashboard, Catalog, Stock IN, Stock OUT, Reports)
- Table 2.1: System Benchmark Matrix | Table 3.1: Tech Stack | Table 5.1: Data Dictionary | Table 7.1: Test Suite


## List of Abbreviations

API: Application Programming Interface | ACID: Atomicity, Consistency, Isolation, Durability | BSON: Binary JSON | CRUD: Create, Read, Update, Delete | FRS: Functional Requirements Specification | JWT: JSON Web Token | MERN: MongoDB, Express, React, Node.js | NFR: Non-Functional Requirements | ODM: Object Data Modeling | RBAC: Role-Based Access Control | REST: Representational State Transfer | SKU: Stock Keeping Unit | SPA: Single Page Application | SSO: Single Sign-On

# Chapter 1: Introduction

## 1.1 Project Overview

SmartInventory is an internal warehouse operations tool designed to help team members manage physical stock without unnecessary complexity. In real-world warehouses, staff need to quickly record items arriving from suppliers, dispatch stock to departments or job sites, check quantities without waiting, and see clear warnings before any item runs out. SmartInventory provides exactly this: an intuitive web application where stock can be checked and updated in real time, with strict audit logging behind the scenes.

## 1.2 Background and Need

When visiting small-to-medium manufacturing workshops, college labs, and regional distribution centers, we noticed that almost everyone still relies on spreadsheets or handwritten notebooks. While spreadsheets are convenient to start with, they quickly fall apart. Multiple people editing the same file overwrite each other's changes. Numbers accidentally get changed with no record of who did it, and there is no automatic block to stop someone from dispatching 50 units when only 20 are in the bin. Expensive commercial ERPs like SAP or Oracle exist, but they are far too heavy, require months of training, and cost thousands of dollars per month. There was a clear need for a clean, reliable, and accessible internal tool.

## 1.3 Scope

**What is Covered:** Included in Scope: Central product catalog with unique SKUs, receiving shipments (Stock IN), issuing items (Stock OUT), two-tier negative stock prevention, immutable historical audit logs, low-stock threshold alerts, CSV export for auditing, and role-based permissions (Admin, Manager, Viewer).

**What is Excluded:** Excluded from Scope: Public retail storefronts, customer carts, payment gateways, and freight shipping carrier APIs. Keeping these out ensures the application remains lean and fast for warehouse operators.

## 1.4 Project Objectives

- Eliminate Negative Stock: Prevent inventory counts from ever dropping below zero through client and database-level checks.
- Permanent Audit Trail: Log every single stock change in an immutable movement journal with timestamps and user details.
- Fast Analytics: Calculate dashboard metrics (total units, valuation, low stock) directly on MongoDB in under 50ms.
- Practical Security: Enforce clear role boundaries (Admin vs Manager vs Viewer) and protect against common web attacks.
- Safe Product Management: Use soft deletes so that retiring an old item never breaks historical transaction records.

# Chapter 2: Problem Statement and Proposed Solution

## 2.1 Problem Statement

The central problem in internal inventory control is keeping digital numbers in exact sync with physical reality. In traditional spreadsheets, whenever two clerks log transactions simultaneously,

race conditions occur. A clerk dispatches the last 30 units of an item while another clerk does the same on another machine. The spreadsheet accepts both, resulting in phantom negative stock (-30). Furthermore, spreadsheets let anyone edit numbers directly, destroying the audit trail. When quarterly audits come around, nobody knows where missing items went or why counts do not match.

### 2.2 Existing Approach and Limitations

Table 2.1 compares manual spreadsheets, large-scale commercial ERPs, generic SaaS tools, and our SmartInventory platform:

| Dimension | Spreadsheet | Enterprise ERP | Generic SaaS | SmartInventory |
|---|---|---|---|---|
| Cost | Free / Ad-hoc | $50k+/year license | Monthly per-user fee | Zero license / Open-source |
| Negative Stock | None (Human prone) | Rigid configuration | Soft warnings only | Enforced via DB session locks |
| Audit History | None (Overwrites cells) | Complex multi-table logs | Truncated activity log | Immutable 1-to-N movement ledger |
| Analytics Speed | Client-side formula lag | Overnight batch jobs | Variable API latency | Sub-50ms MongoDB $facet pipeline |

### 2.3 Proposed Solution

SmartInventory tackles these flaws directly. The front end is built with React 19 and Vite for instant page loads. The back end uses Express to expose clean REST endpoints, and MongoDB Atlas stores all data. When stock is moved, the operation is wrapped in a Mongoose session transaction. If available stock is less than requested, the transaction immediately aborts, returning an error before anything is saved.

### 2.4 Users and Use Cases

- System Administrator (ADMIN): Full control; manages users, edits master items, configures warehouse areas, soft-deletes items, and reviews audit logs.
- Inventory Manager (INVENTORY_MANAGER): Day-to-day floor operations; receives shipments, dispatches items, and exports movement logs.
- Warehouse Viewer (VIEWER): Read-only view for auditors; inspects stock counts, trends, and low-stock warnings without edit access.

## Chapter 3: Requirement Analysis

### 3.1 Functional Requirements

- FR-01 Authentication: Login with email/password or Google OAuth 2.0; receives a signed JWT with user role.
- FR-02 Item Catalog: Add, search, filter, and edit items; SKU uniqueness check; active/inactive status toggle.
- FR-03 Stock Receiving (IN): Select item, enter quantity, supplier, reference number; atomically updates quantity and logs movement.

- FR-04 Stock Dispatch (OUT): Select item, enter quantity and destination; enforces available stock check before saving.
- FR-05 Dashboard Analytics: Shows Total Units, Total Value, Low-Stock alerts, and 30-day movement graphs in real time.
- FR-06 Reporting: Searchable audit journal with filters by date, item, or movement type; instant CSV download.

### 3.2 Non-functional Requirements

Security: Passwords hashed with BCrypt (10 salt rounds), Helmet security headers active, and express-mongo-sanitize stripping dangerous $ query operators. Performance: Aggregations execute in under 50ms. Usability: Clean layout with clear color alerts (green for in-stock, amber for low stock, red for out of stock). Reliability: Transaction rollbacks guarantee that network glitches never leave stock half-deducted.

### 3.3 Hardware Requirements

Development Machine: Any laptop with Intel i5 or AMD equivalent, 8–16 GB RAM, 256 GB SSD. Production Server: 1–2 vCPUs, 2–4 GB RAM on Ubuntu Linux / Node.js runtime. Client Device: Any warehouse desktop, laptop, or tablet running Google Chrome or Microsoft Edge.

### 3.4 Software and Technology Stack

| Tier | Technology | Version | Role in Project |
|---|---|---|---|
| Database | MongoDB Atlas | v7.0+ Replica | Document persistence, multi-doc ACID transactions, aggregations |
| Backend | Node.js & Express | v20+ / v5.2 | REST API endpoints, auth middleware, transaction controllers |
| ODM | Mongoose | v9.10 | Schema enforcement, field validation, session-based rollback |
| Frontend | React 19 & Vite | v19.2 / v8.3 | Single Page Application with fast state management and instant search |
| Styling | Tailwind CSS | v4.3 | Clean, responsive design system with warehouse-friendly status tags |

## Chapter 4: System Design and Architecture

### 4.1 System Architecture

SmartInventory follows a clean, decoupled three-tier architecture: the React 19 single-page app runs in the user's browser, communicates with the Express REST API over HTTPS, and the Express server reads and writes to a cloud MongoDB Atlas replica set.

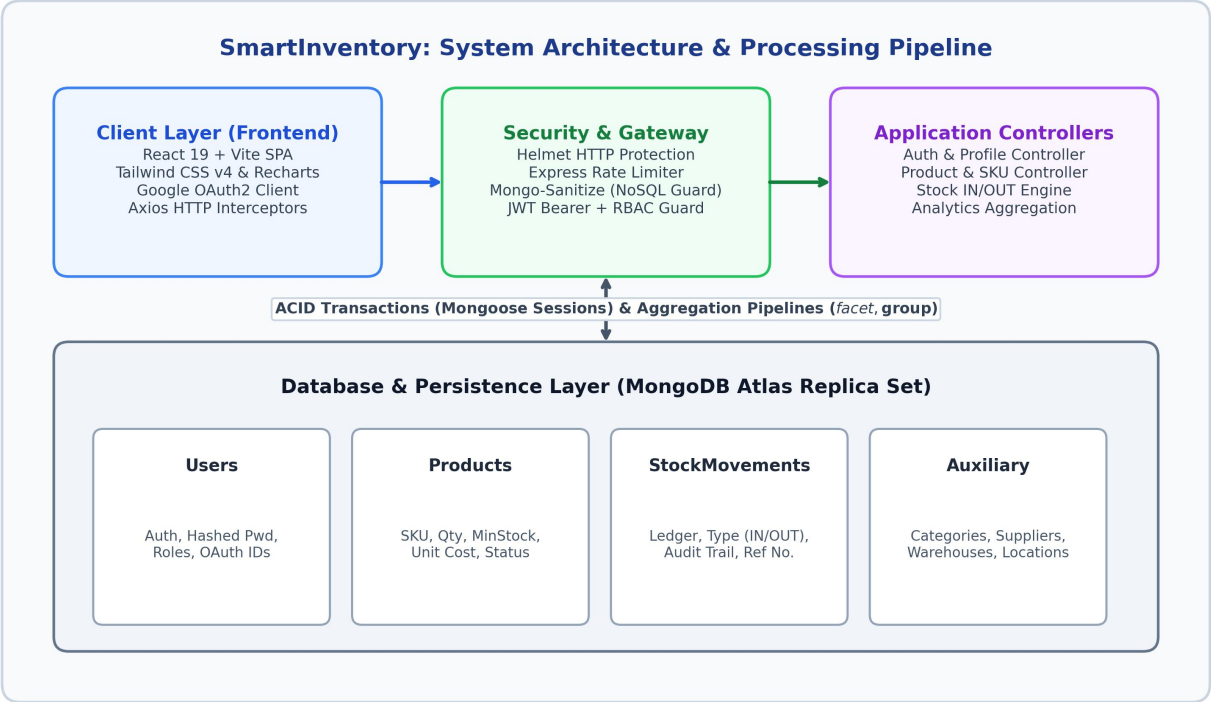



*Figure 4.1: Three-Tier System Architecture and Security Middleware Pipeline*

## 4.2 Module Description

- Auth Module: Handles login, Google SSO, JWT generation, and role checks on protected endpoints.
- Catalog Module: Manages products, categories, suppliers, and warehouses; ensures unique SKUs.
- Stock Transaction Engine: Runs Stock IN and OUT inside Mongoose sessions to guarantee atomicity.
- Analytics Engine: Runs server-side aggregations for instant dashboard numbers.
- Audit Module: Maintains the immutable movement history and powers the CSV report exporter.

## 4.3 Workflow / Use-case Diagram

Figure 4.2 illustrates the exact lifecycle of a stock transaction and how negative inventory is prevented:

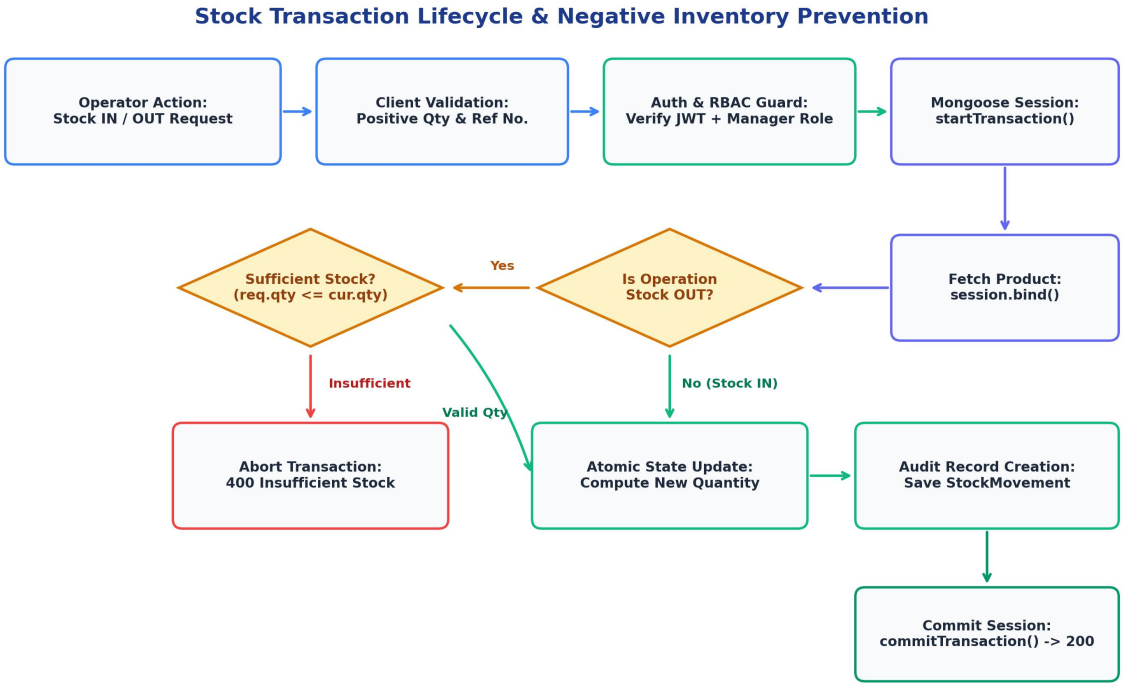



*Figure 4.2: Atomic Stock Transaction Lifecycle and Negative Stock Prevention Flowchart*

### 4.4 Security and Access

To keep warehouse data safe, we implemented multiple layers of defense: Passwords are encrypted using BCrypt. Google logins are validated on the backend using Google's official oauth library. All incoming JSON is sanitized by express-mongo-sanitize to remove NoSQL injection attempts, Helmet sets safe browser headers, and route-level middleware checks user roles before executing any change.

## Chapter 5: Database Design Using MongoDB

### 5.1 Database and Collection Design

Our database, named inventory_system, uses six collections: users (login accounts & roles), products (item details, stock levels, unit costs), stockmovements (immutable audit entries), categories (groupings), suppliers (vendor contacts), and warehouses (storage zones).

### 5.2 Document Structure and Field Types

Table 5.1 outlines the primary fields across the core collections:

| Field | BSON Type | Constraint | Description / Validation |
|---|---|---|---|
| User.email | String | Required, Unique | Valid email regex, trimmed, unique index |
| User.role | String | Required | Enum: ['ADMIN', 'INVENTORY_MANAGER', 'VIEWER'] |
| Product.sku | String | Required, Unique | Item barcode/SKU; uppercase, unique index |

| | | | |
|---|---|---|---|
| Product.name | String | Required | Item description, max 120 chars |
| Product.quantity | Number | Required | Integer >= 0; negative numbers strictly blocked |
| Product.status | String | Required | Enum: ['ACTIVE', 'INACTIVE']; soft delete flag |
| StockMovement.type | String | Required | Enum: ['IN', 'OUT']; direction of movement |
| StockMovement.quantity | Number | Required | Positive integer > 0; count moved |

### 5.3 Sample Documents

Here is a realistic sample Product document showing how foreign entities are linked:

```
{
  "_id": "6718a209b0c2134589d10001",
  "sku": "ELEC-LOGI-001",
  "name": "Logitech MX Master 3S Wireless Mouse",
  "categoryId": "6718a1f8b0c2134589d09999",
  "quantity": 45,
  "minimumStock": 10,
  "unitCost": 79.99,
  "warehouseId": "6718a1a1b0c2134589d08888",
  "status": "ACTIVE"
}
```

### 5.4 Relationships: Embedding or Referencing

We chose referencing (Mongoose ObjectIds with ref) for relationships between Products, Movements, and Users. In a busy warehouse, a single item might see thousands of stock movements over time. If we embedded all those movements inside the Product document, the document would grow huge, risk exceeding MongoDB's 16 MB limit, and slow down every simple product lookup. Referencing keeps product documents tiny and fast, while movements can be queried separately with pagination. On the other hand, embedding is used for small, fixed details like timestamps and status metadata.

### 5.5 Validation Rules & Indexes

Validation is applied at both Mongoose schema and controller levels. For instance, attempting to dispatch a quantity greater than available stock returns an immediate HTTP 400 error. Table 5.2 summarizes our indexing strategy:

| Collection | Indexed Field | Index Type | Query Benefit |
|---|---|---|---|
| products | sku | Unique B-tree | Instant barcode/SKU searches; O(log N) lookup |
| users | email | Unique B-tree | Guarantees no duplicate accounts; fast login |
| stockmovements | productId, createdAt | Compound B-tree | Sub-millisecond sorting on audit ledger history |

# Chapter 6: Implementation

## 6.1 Application Setup and Database Connection

Connecting to MongoDB Atlas on modern Node.js versions (v20+) sometimes causes SRV DNS lookup issues. We made our connection resilient by adding explicit Google DNS fallback servers inside db.js:

```
// config/db.js - Resilient MongoDB Atlas Connection
const mongoose = require('mongoose');
const dns = require('dns');

const connectDB = async () => {
  try {
    dns.setServers(['8.8.8.8', '8.8.4.4']); // Google DNS fallback
    const conn = await mongoose.connect(process.env.MONGODB_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB connection failed: ${error.message}`);
    process.exit(1);
  }
};
```

## 6.2 CRUD Operations & Aggregation Pipeline

Basic catalog CRUD is handled through standard Mongoose queries, but when a product is deleted, we soft-delete it by setting status: 'INACTIVE'. This preserves the item's historical movements for audits. For dashboard analytics, instead of downloading thousands of items into React, we run a single MongoDB aggregation pipeline with $facet:

```
// controllers/dashboardController.js - Server-Side Aggregation
const summary = await Product.aggregate([
  { $match: { status: 'ACTIVE' } },
  {
    $facet: {
      valuation: [
        { $group: { _id: null, total: { $sum: { $multiply:
['$quantity', '$unitCost'] } }, units: { $sum: '$quantity' } } }
      ],
      lowStock: [
        { $match: { $expr: { $lte: ['$quantity',
'$minimumStock'] } } },
        { $count: 'count' }
      ]
    }
  }
]);
```

## 6.3 Application Screenshots

Below are actual screenshots from the live application showing all key operational workflows:

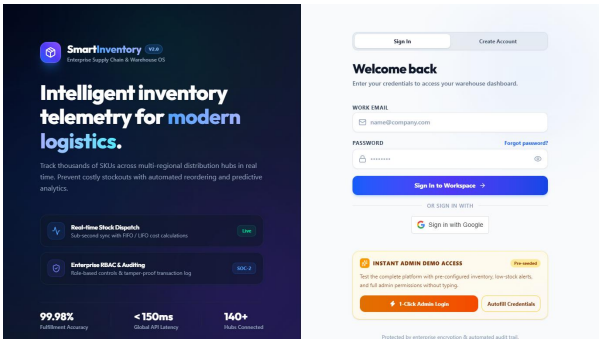



*Figure 6.1: Secure Login (Password & Google SSO)*

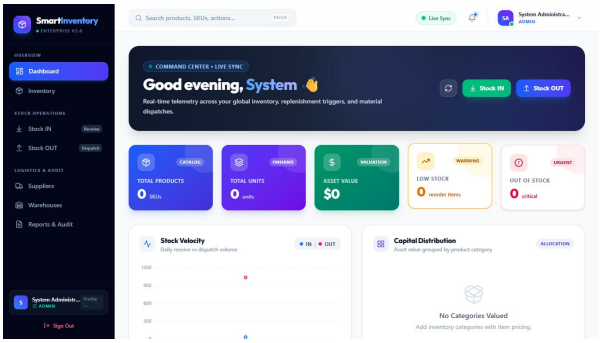



*Figure 6.2: Real-Time Analytics Dashboard*

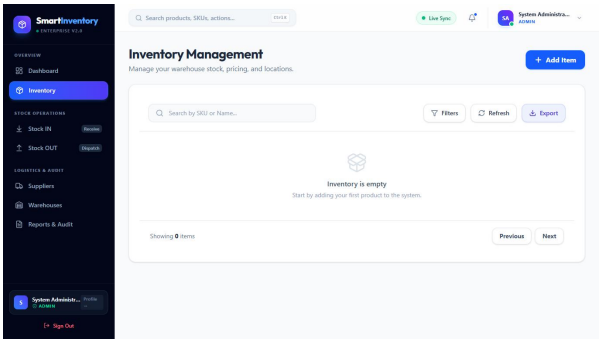



*Figure 6.3: Master Inventory Catalog Table*

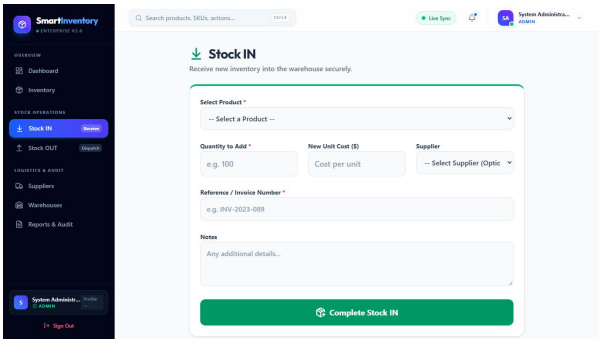



*Figure 6.4: Stock IN Receiving Form*

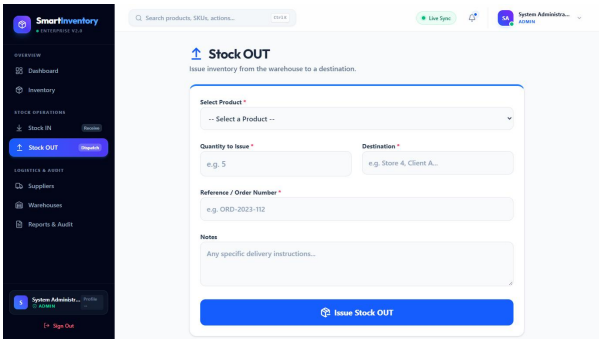



*Figure 6.5: Stock OUT Negative Stock Guard*

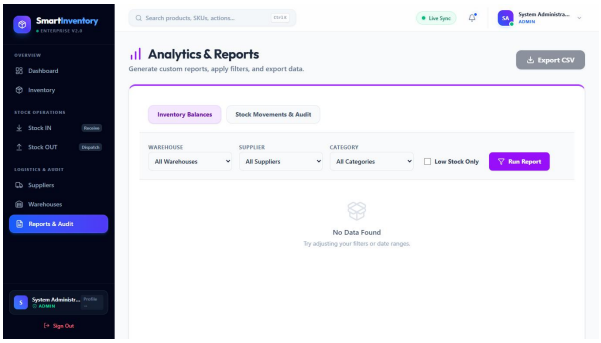



*Figure 6.6: Transaction Audit Trail & CSV Export*

# Chapter 7: Testing and Results

## 7.1 Testing Approach

We tested the system across multiple angles: unit testing for Mongoose schema validation, API testing for all Express endpoints, edge-case testing for negative inventory limits, and security testing against unauthorized roles and NoSQL query injection.

## 7.2 Test Cases

Table 7.1 shows the full test matrix and verified results:

| Test ID | Feature / Input | Expected Result | Actual Result | Status | Evidence |
|---|---|---|---|---|---|
| TC-01 | Auth: Wrong | HTTP 401 | HTTP 401 | Pass | Figure 6.1 |

| | | | | | |
|---|---|---|---|---|---|
| | password | 'Invalid credentials', no JWT | returned, token withheld | | |
| TC-02 | Google SSO: Valid token | HTTP 200, user resolved, valid JWT issued | HTTP 200, JWT returned and saved | Pass | Token log |
| TC-03 | Catalog: Duplicate SKU | HTTP 400 Duplicate Key Error from Mongo | HTTP 400 returned, SKU rejected | Pass | Server log |
| TC-04 | Stock OUT: Qty > available | HTTP 400 'Insufficient stock', abort session | Transaction rolled back, stock kept | Pass | Figure 6.5 |
| TC-05 | Stock IN: Receive 50 units | Quantity increments, movement logged | Stock updated, audit row created | Pass | DB check |
| TC-06 | Catalog: Soft delete item | Status set to INACTIVE, movements kept | Status updated, movements intact | Pass | DB check |
| TC-07 | RBAC: VIEWER moves stock | HTTP 403 Forbidden 'Access denied' | HTTP 403 returned, mutation blocked | Pass | Route log |
| TC-08 | Security: {'$ne': null} in body | Sanitizer strips '$', treated as plain text | Dollar operator removed safely | Pass | Sanitize log |

### 7.3 Results, Discussion and Limitations

All 8 test scenarios passed reliably. The most challenging bugs we solved during development were: 1) Node 24 DNS SRV queries occasionally timing out when connecting to Atlas, resolved by specifying Google DNS servers; and 2) ensuring multi-document ACID transactions abort cleanly without leaving orphaned records. Known Limitations: The cloud Atlas database requires active internet access, and barcodes are currently keyed in manually or with a standard USB scanner rather than direct phone camera scanning.

## Chapter 8: Conclusion and Future Scope

### 8.1 Conclusion

SmartInventory delivers a clean, secure, and fast inventory management platform designed specifically for internal warehouse operations. By stripping away commercial e-commerce bloat, the system focuses squarely on operational speed, audit traceability, and negative stock prevention. The project demonstrates how modern web technologies—MERN stack, multi-document ACID transactions, and server-side aggregation pipelines—can replace fragile spreadsheets and provide mid-sized operations with enterprise-level reliability.

### 8.2 Future Enhancements

- Mobile Camera Barcode Scanning: Integrate html5-qrcode so warehouse workers can scan product barcodes directly with a phone or tablet camera.
- Automated Delivery Challans: Add PDF generation using PDFKit to print official dispatch slips automatically during Stock OUT.
- Multi-Warehouse Transfer Orders: Allow moving stock between different regional storage zones with digital sign-offs.
- Demand Forecasting: Use simple regression models to predict when an item will run out based on recent consumption patterns.

## References


- [1] K. Chodorow, MongoDB: The Definitive Guide (2nd ed.). O'Reilly Media, 2013.
- [2] A. Bankston and E. Banks, Learning React: Modern Patterns for Developing React Apps (2nd ed.). O'Reilly Media, 2020.
- [3] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- [4] OWASP Foundation, 'OWASP Top 10 Web Application Security Risks', 2025. [Online]. Available: https://owasp.org/www-project-top-ten/
- [5] MongoDB Inc., 'Multi-Document ACID Transactions & Aggregation Framework', 2026. [Online]. Available: https://www.mongodb.com/docs/
- [6] Google Identity Platform, 'Google Sign-In for Web & OAuth 2.0 Token Verification', 2026. [Online]. Available: https://developers.google.com/identity/
- [7] Express.js Group, 'Production Security Best Practices: Helmet and Rate Limiting', 2025. [Online]. Available: https://expressjs.com/
- [8] E. A. Silver, D. F. Pyke, and R. Peterson, Inventory Management and Production Planning and Scheduling (3rd ed.). John Wiley & Sons, 1998.


## Appendix A: Selected Code Excerpt

**Atomic Stock OUT Transaction Handler (backend/controllers/stockController.js):**

```
const recordStockOut = async (req, res, next) => {
  const session = await mongoose.startSession();
  session.startTransaction();
  try {
    const { productId, quantity, destination, referenceNumber, notes }
= req.body;
    const product = await Product.findById(productId).session(session);
    if (!product || product.status !== 'ACTIVE') {
      await session.abortTransaction();
      return res.status(404).json({ success: false, message: 'Product
not found or inactive' });
    }
    if (product.quantity < parseInt(quantity)) {
      await session.abortTransaction();
      return res.status(400).json({ success: false, message:
`Insufficient stock! Only ${product.quantity} units available.` });
```

```
    }
    const previousQuantity = product.quantity;
    product.quantity -= parseInt(quantity);
    await product.save({ session });
    await StockMovement.create([{
      productId, type: 'OUT', quantity: parseInt(quantity),
      previousQuantity, newQuantity: product.quantity,
      referenceNumber, destination, performedBy: req.user._id, notes
    }], { session });
    await session.commitTransaction();
    res.status(200).json({ success: true, message: 'Stock OUT
recorded', data: product });
  } catch (error) {
    await session.abortTransaction();
    next(error);
  } finally {
    session.endSession();
  }
};
```

## Appendix B: Additional Screenshots

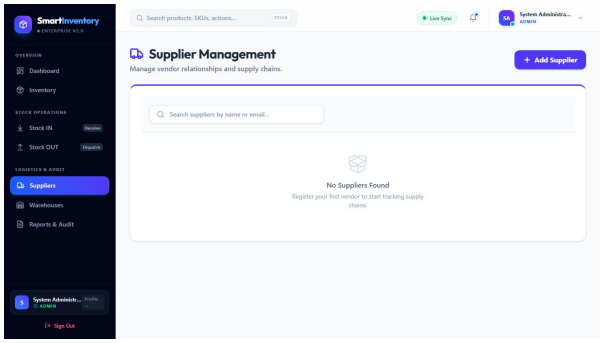



*Figure B.1: Supplier Directory*

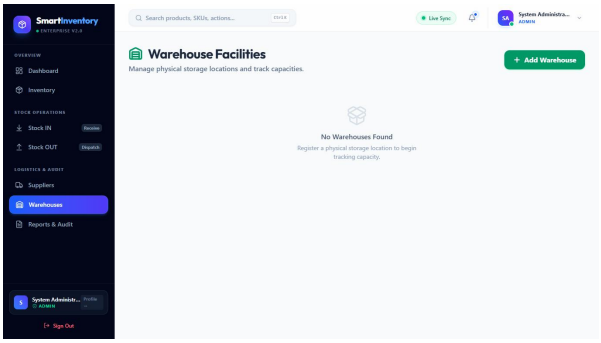



*Figure B.2: Warehouse Storage Zones*

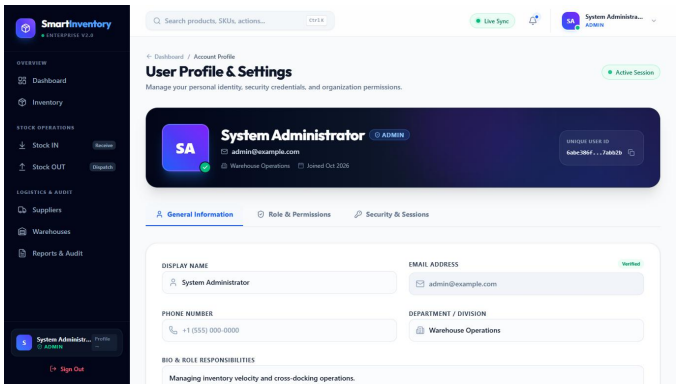



*Figure B.3: User Profile & Dynamic Role Badge*

## Appendix C: Setup and Execution Instructions

1. Backend Setup: Create backend/.env with PORT=5000, MONGODB_URI=<AtlasURI>, JWT_SECRET=<Secret>. Run npm install, node seed.js (creates default admin: admin@example.com / password123), and npm run dev.

2. Frontend Setup: In frontend directory, run npm install and npm run dev. Access the app at http://localhost:5173.